

⚠️ PROBLEMAS COMUNES & IDEA FINAL

🔥 ERRORES FRECUENTES

ERROR	CONSECUENCIA
Escribir SQL en lugar de JPQL	Error en tiempo de ejecución
Usar nombre de tabla en vez de entidad	<code>usuarios</code> → debe ser <code>Usuario</code>
Concatenar valores en el string	SQL injection, errores difíciles
No usar <code>TypedQuery</code>	Casting peligroso en runtime
<code>getSingleResult()</code> sin resultado	<code>NoResultException</code>
JOIN sobre columna, no sobre relación	JPQL no conoce columnas

✅ BUENAS PRÁCTICAS

- Usa siempre `TypedQuery` para tipado seguro
- Usa **parámetros nombrados** (`:nombre`) nunca concatenes
- Trabaja con **entidades y atributos**, no tablas ni columnas
- Usa `NamedQuery` para consultas reutilizables
- Prefiere **JPQL** para consultas habituales; Critería para dinámicas
- Cierra siempre `EntityManager` y `EMFactory`

🔍 LO QUE HIBERNATE HACE POR TI

Convierte JPQL en SQL real automáticamente:

```
// JPQL que escribes:
SELECT u FROM Usuario u
WHERE u.nombre = :nombre
```

```
// SQL que genera Hibernate:
SELECT id, nombre, email
FROM usuarios
WHERE nombre = 'Ana'
```

- Mapea objetos resultado automáticamente
- Gestiona el cache de primer nivel
- Optimiza consultas según el dialecto de BD

💡 Idea final

JPA no elimina las consultas, las *transforma*. Se sigue necesitando entender cómo funcionan, pero ahora se trabaja con entidades y relaciones en lugar de tablas y joins manuales.

🌟 NAMEDQUERY & CRITERIA API

📄 NAMEDQUERY — CONSULTAS REUTILIZABLES

Se definen **en la entidad** con `@NamedQuery` y se invocan por nombre.

```
@Entity
@NamedQuery(
    name = "Usuario.porNombre",
    query = "SELECT u FROM Usuario u "
        + "WHERE u.nombre = :nombre"
)
public class Usuario { ... }
```

```
// Invocación:
TypedQuery<Usuario> q =
    em.createNamedQuery(
        "Usuario.porNombre", Usuario.class);
q.setParameter("nombre", "Ana");
List<Usuario> lista = q.getResultList();
```

✓ Reutilización ✓ Centralización ✓ Validada al arrancar

⚙️ CRITERIA API — CONSULTAS PROGRAMÁTICAS

Construye consultas **sin strings**. Ideal para filtros dinámicos.

```
// Listar todos los usuarios
CriteriaBuilder cb =
    em.getCriteriaBuilder();
CriteriaQuery<Usuario> cq =
    cb.createQuery(Usuario.class);
Root<Usuario> root =
    cq.from(Usuario.class);
cq.select(root);
```

```
TypedQuery<Usuario> q =
    em.createQuery(cq);
List<Usuario> lista = q.getResultList();
```

```
// Con filtro dinámico por nombre
cq.select(root).where(
    cb.equal(root.get("nombre"), "Ana")
);
```

JPQL VS CRITERIA API

	JPQL	CRITERIA API
Sintaxis	String legible	Código Java
Errores	Runtime	Compilación
Dinámico	Difficil	✓ Ideal
Lectura	✓ Fácil	Verboso
Uso habitual	✓ Sí	Casos especiales

DAM · PROGRAMACIÓN · JPA / ORM



Consultas con JPA / Hibernate

JPQL · TypedQuery · JOIN · NamedQuery · Criteria

JPQL TypedQuery :parámetros JOIN @NamedQuery Criteria API

SELECT parcial

FLUJO DE UNA CONSULTA

JPQL → Hibernate → SQL → Objetos

🔍 JPQL – FUNDAMENTOS

🚫 LIMITACIÓN DE FIND()

```
Usuario u = em.find(Usuario.class, 1);  
// Solo busca por clave primaria
```

En aplicaciones reales se necesita buscar por nombre, filtrar por fechas, obtener listas, combinar entidades...

📖 JPQL VS SQL

JPQL es orientado a **objetos**; SQL a **tablas**.

	SQL	JPQL
Trabaja con	tablas / columnas	entidades / atributos
Ejemplo FROM	FROM usuarios	FROM Usuario u
Ejemplo WHERE	nombre='Ana'	u.nombre = :n
Resultado	filas / columnas	objetos Java
<pre>-- SQL: SELECT * FROM usuarios WHERE nombre = 'Ana' // JPQL: SELECT u FROM Usuario u WHERE u.nombre = 'Ana'</pre>		

QUERY BÁSICA

```
Query q = em.createQuery(  
    "SELECT u FROM Usuario u");  
List<Usuario> lista = q.getResultList();
```

✅ TYPEDQUERY – FORMA RECOMENDADA

Evita casting. Tipado seguro en tiempo de compilación.

```
TypedQuery<Usuario> q =  
    em.createQuery(  
        "SELECT u FROM Usuario u",  
        Usuario.class  
    );  
List<Usuario> lista = q.getResultList();
```

✓ Sin casting ✓ Código limpio ✓ Tipado seguro

RESULTADO ÚNICO

```
Usuario u = q.getSingleResult();  
// ⚠ lanza NoResultException si no hay  
// ⚠ lanza NonUniqueResultException si hay +1
```

★ PARÁMETROS, JOIN & SELECT PARCIAL

🔒 PARÁMETROS NOMBRADOS – SIEMPRE

❌ Nunca: "... WHERE nombre = '" + nombre + "'" → SQL injection

```
TypedQuery<Usuario> q =  
    em.createQuery(  
        "SELECT u FROM Usuario u "  
        + "WHERE u.nombre = :nombre",  
        Usuario.class  
    );  
q.setParameter("nombre", "Ana");  
List<Usuario> r = q.getResultList();
```

CONDICIONES & LIKE

```
// Mayor que  
SELECT p FROM Producto p  
WHERE p.precio > :precio  
  
// LIKE (emails de gmail)  
SELECT u FROM Usuario u  
WHERE u.email LIKE :patron  
// setParameter("patron", "%gmail.com")  
  
// Fecha  
SELECT p FROM Pedido p  
WHERE p.fecha = :fecha
```

🔗 JOIN CON RELACIONES – EL PODER DEL ORM

El JOIN se hace sobre la **relación del objeto**, no sobre columnas.

```
// Pedidos de un usuario por nombre  
TypedQuery<Pedido> q =  
    em.createQuery(  
        "SELECT p FROM Pedido p "  
        + "JOIN p.usuario u "  
        + "WHERE u.nombre = :nombre",  
        Pedido.class  
    );  
q.setParameter("nombre", "Ana");
```

💡 `p.usuario` es el *atributo* de la entidad, no la columna `usuario_id`.

SELECT PARCIAL – CAMPOS CONCRETOS

```
// Un campo → List<String>  
SELECT u.nombre FROM Usuario u  
  
// Varios campos → List<Object[]>  
SELECT u.nombre, u.email FROM Usuario u
```

⚠ No devuelve objetos completos. Castea con cuidado.

📄 MÉTODOS CLAVE & EJERCICIOS

🔍 MÉTODOS DE CONSULTA

MÉTODO	DEVUELVE	USO
createQuery(jpql, T.class)	TypedQuery<T>	JPQL tipado
createQuery(jpql)	Query	Sin tipo
createNamedQuery(name, T.class)	TypedQuery<T>	Named
getResultList()	List<T>	Varios resultados
getSingleResult()	T	Uno solo
setParameter(k, v)	Query	Parámetros
setMaxResults(n)	Query	Limitar filas
setFirstResult(n)	Query	Paginación

📄 PAGINACIÓN

```
q.setFirstResult(0); // página 1  
q.setMaxResults(10); // 10 por página
```

📄 EJERCICIOS DEL BLOQUE

#	TAREA	TIPO
1	Buscar usuarios por nombre	JPQL
2	Productos con precio mayor a X	WHERE
3	Pedidos por fecha	WHERE
4	JOIN usuario → pedidos	JOIN
5	JOIN pedido → productos	JOIN
6	Solo nombres de usuarios	parcial
7	Emails con LIKE (%gmail.com)	LIKE
8	NamedQuery en Usuario + ejecución	Named
9	Critería API: listar todos los usuarios	Critería
10	Critería API: filtro dinámico por nombre	Critería